

Variables, tipos y operadores

Presentación teórica · Módulo 2

```
int edad = 35;  
double precio = 19.95;  
boolean activo = true;  
char categoria = 'A';  
String nombre =  
"Lucía";  
final double IVA =  
0.21;
```

Objetivos del módulo

Qué debe dominar el alumnado al terminar este bloque.

Declarar variables y constantes con sintaxis correcta.
Elegir tipos adecuados según el dato que se quiere guardar.
Aplicar operadores aritméticos, de comparación y lógicos.
Leer datos desde teclado con Scanner y convertirlos.
Resolver pequeños problemas de cálculo en consola.

// Al final del bloque el alumnado debe poder explicar:

```
String nombre = "Ana";  
int edad = 30;  
double precio = 19.95;  
final double IVA = 0.21;  
  
double total = precio + precio * IVA;
```

Idea docente: no basta con que el programa funcione; deben poder modificar valores, tipos y nombres sin romper la sintaxis.

Qué es una variable

Una variable es un espacio donde guardamos un dato.

Tiene un nombre para poder usarla después.

Tiene un tipo, porque Java necesita saber qué clase de dato almacena.

Su valor puede cambiar durante la ejecución del programa.

```
int edad = 35;
```

```
// edad es el nombre
```

```
// int es el tipo
```

```
// 35 es el valor
```

```
edad = 36; // cambia el valor
```

Comparación sencilla: una variable es como una caja etiquetada donde guardamos un dato concreto.

Estructura de una declaración

```
tipo nombre = valor;  
  
int cantidad = 4;  
double precio = 12.50;  
String producto = "Teclado";  
boolean oferta = true;
```

Primero se escribe el tipo de dato.
Después se escribe el nombre de la variable.
El signo = asigna un valor.
La instrucción termina con punto y coma ;

El signo = no significa “igual que” como en matemáticas: en programación significa “guardar este valor en esta variable”.

Tipos básicos más usados

Números y valores simples

int: números enteros.
long: enteros muy grandes.
double: números decimales.
boolean: true o false.
char: un único carácter.

Texto y constantes

String: cadenas de texto.
final: convierte una variable en constante.
Cada tipo limita qué valores se pueden guardar.
Elegir bien el tipo evita errores posteriores.

En Java el tipo se indica siempre al declarar la variable. Esto ayuda a detectar errores antes de ejecutar.

int y long: números enteros

int se usa para cantidades sin decimales.

Sirve para edades, unidades, puntos, contadores, años o stock.

No admite coma decimal.

long se reserva para enteros muy grandes.

```
int edad = 35;  
int cantidad = 4;  
int puntos = 10;  
int stock = 25;  
long poblacion = 8000000L;  
  
System.out.println(edad);
```

Error frecuente: escribir `int precio = 12.50;` no vale porque 12.50 es decimal.

double: números decimales

Se usa para precios, alturas, medias, pesos o temperaturas.

Permite guardar valores con decimales.

En Java se usa punto decimal, no coma.

Es ideal para cálculos con importes o medidas.

```
double precio = 19.95;  
double altura = 1.68;  
double media = 7.5;  
double celsius = 25.0;  
  
System.out.println(precio);
```

En Java se escribe 1.68, no 1,68. La coma produciría un error de sintaxis.

boolean: verdadero o falso

Solo puede guardar dos valores: true o false.
Sirve para representar estados o respuestas sí/no.
Más adelante se usa mucho en condicionales.
No se escribe entre comillas.

```
boolean activo = true;  
boolean aprobado = false;  
boolean tieneCarnet = true;  
boolean estaEnOferta = false;  
  
System.out.println(activo);
```

true y false son palabras reservadas de Java; no son textos. Por eso no llevan comillas.

char: un solo carácter

Guarda un único carácter.

Se escribe entre comillas simples.

Puede ser una letra, un número como carácter o un símbolo.

No es lo mismo que String.

```
char categoria = 'A';  
char inicial = 'P';  
char simbolo = '#';  
  
System.out.println(categoria);  
  
// String texto = "A";
```

char usa comillas simples: 'A'. String usa comillas dobles: "A".

String: texto

String permite guardar texto.

Se escribe siempre entre comillas dobles.

Aunque se usa como un tipo básico, realmente es una clase.

Permite concatenar texto con variables usando +.

```
String nombre = "Lucía";  
String ciudad = "Gijón";  
String curso = "Java desde cero";  
  
System.out.println("Nombre: " +  
nombre);
```

Error frecuente: olvidar las comillas dobles al guardar texto.

Constantes con final

Una constante es un valor que no debe cambiar.

En Java se crea usando final.

Se suele usar para IVA, descuentos fijos, límites o valores matemáticos.

Por convención, muchas constantes se escriben en mayúsculas.

```
final double IVA = 0.21;  
final int MAYORIA_EDAD = 18;  
  
double precio = 100;  
double total = precio + precio *  
IVA;  
  
System.out.println(total);
```

Si intentamos cambiar una constante después, Java marcará error.

Nombres de variables

Buenas prácticas

Usar nombres descriptivos.
Empezar en minúscula.
Usar camelCase si hay varias palabras.
Evitar abreviaturas confusas.

Ejemplos

nombreAlumno
precioProducto
numeroUnidades
estaAprobado
totalFactura

Un buen nombre ayuda a entender el programa sin tener que adivinar qué representa cada dato.

Asignar y cambiar valores

Una variable puede declararse con un valor inicial.
Después se puede cambiar su valor, siempre que siga siendo del mismo tipo.
El nuevo valor sustituye al anterior.

```
int puntos = 10;  
System.out.println(puntos);  
  
puntos = 20;  
System.out.println(puntos);  
  
// puntos = "veinte"; // error
```

Este concepto suele costar: la variable conserva el último valor asignado.

Mostrar variables en pantalla

`System.out.println()` muestra información en consola.
Podemos mostrar texto literal entre comillas.
También podemos mostrar variables.
Con `+` unimos texto y valores.

```
String producto = "Teclado";  
double precio = 29.99;  
  
System.out.println(producto);  
System.out.println(precio);  
System.out.println("Producto: " +  
    producto);  
System.out.println("Precio: " + precio);
```

El `+` aquí no siempre suma: cuando hay texto, concatena.

Operadores aritméticos

+ suma

- resta

* multiplica

/ divide

% obtiene el resto de una división

```
int a = 10;  
int b = 3;
```

```
System.out.println(a + b); // 13  
System.out.println(a - b); // 7  
System.out.println(a * b); // 30  
System.out.println(a / b); // 3  
System.out.println(a % b); // 1
```

El operador % es muy útil para saber si un número es par o impar, pero eso se verá mejor con condicionales.

División y decimales

Si dividimos enteros, Java puede devolver un entero.
Para obtener decimales, al menos uno de los valores debe ser double.
Esto es importante al calcular medias.

```
int a = 5;  
int b = 2;  
  
System.out.println(a / b); // 2  
  
double resultado = 5.0 / 2.0;  
System.out.println(resultado); // 2.5
```

Buen ejemplo para clase: calcular una media usando / 3.0 si queremos asegurar resultado decimal.

Prioridad de operaciones

Java respeta la prioridad matemática.

Primero multiplicación, división y resto.

Después suma y resta.

Los paréntesis cambian el orden y mejoran la claridad.

```
double resultado1 = 10 + 5 * 2;  
// resultado1 vale 20
```

```
double resultado2 = (10 + 5) * 2;  
// resultado2 vale 30
```

```
System.out.println(resultado1);  
System.out.println(resultado2);
```

Aunque Java sepa resolver la prioridad, es buena práctica usar paréntesis cuando mejora la lectura.

Operadores de comparación

Sirven para comparar valores.

El resultado siempre es boolean: true o false.

Se usan mucho en condicionales.

No confundir = con ==.

```
int edad = 20;

System.out.println(edad > 18); // true
System.out.println(edad < 18); // false
System.out.println(edad == 20); // true
System.out.println(edad != 20); // false
```

= asigna un valor. == compara si dos valores son iguales.

Operadores lógicos

&& significa “y”: deben cumplirse las dos condiciones.

|| significa “o”: basta con que se cumpla una.

! niega una condición.

También producen true o false.

```
int edad = 20;  
boolean tieneEntrada = true;  
  
boolean puedeAcceder = edad >= 18 &&  
tieneEntrada;  
  
System.out.println(puedeAcceder);
```

Aunque se estudian mejor con if, conviene presentarlos aquí porque forman parte de los operadores.

Entrada de datos con Scanner

Scanner permite leer datos escritos por teclado.
Necesita import java.util.Scanner; al principio del archivo.
Creamos un objeto Scanner conectado a System.in.
Para principiantes, conviene leer con nextLine().

```
import java.util.Scanner;

public class Entrada {
    public static void main(String[] args) {
        Scanner teclado = new
Scanner(System.in);

        System.out.print("Nombre: ");
        String nombre = teclado.nextLine();
    }
}
```

Scanner convierte los programas en interactivos: el usuario introduce datos durante la ejecución.

Convertir texto a números

nextLine() siempre lee texto.

Para convertir a entero usamos Integer.parseInt().

Para convertir a decimal usamos Double.parseDouble().

Si el texto no es válido, aparecerá

NumberFormatException.

```
Scanner sc = new Scanner(System.in);

System.out.print("Edad: ");
int edad =
Integer.parseInt(sc.nextLine());

System.out.print("Precio: ");
double precio =
Double.parseDouble(sc.nextLine());
```

Según el manual, leer con nextLine() y convertir números evita problemas con saltos de línea pendientes.

Comparar textos correctamente

Para comparar números usamos ==.

Para comparar el contenido de un String usamos equals().
== en String puede dar resultados inesperados.
equalsIgnoreCase() ignora mayúsculas y minúsculas.

```
String ciudad = "Gijón";

System.out.println(ciudad.equals("Gijón")
);
System.out.println(ciudad.equalsIgnoreCase("GIJÓN"));

// Evitar:
// ciudad == "Gijón"
```

Esta regla conviene repetirla desde el principio: los textos se comparan con equals(), no con ==.

Ejemplo guiado: factura con IVA

Pedir precio y cantidad al usuario.
Calcular subtotal.
Calcular IVA del 21 %.
Calcular total sumando subtotal e IVA.
Mostrar resultados formateados.

```
Scanner sc = new Scanner(System.in);

System.out.print("Precio: ");
double precio = Double.parseDouble(sc.nextLine());

System.out.print("Cantidad: ");
int cantidad = Integer.parseInt(sc.nextLine());

double subtotal = precio * cantidad;
double iva = subtotal * 0.21;
double total = subtotal + iva;

System.out.printf("Total: %.2f €\n", total);
```

Errores frecuentes del bloque

Errores de sintaxis

Olvidar el punto y coma.
No cerrar llaves.
Usar coma decimal en vez de punto.
No coincidir clase y archivo.

Errores de variables

Cambiar una variable a otro tipo.
Usar una variable no declarada.
Comparar String con ==.
Convertir mal un dato escrito por teclado.

Conviene mostrar errores reales en directo y explicar el mensaje antes de corregirlo.

Cierre y comprobación

Sé declarar variables con tipos adecuados.
Sé diferenciar String, int, double, boolean, char y long.
Sé usar final para constantes.
Sé aplicar +, -, *, /, %.
Sé pedir datos con Scanner y convertirlos.
Sé explicar qué hace cada línea de un ejercicio sencillo.

Reto final:

Crea Factura.java
Pide precio y cantidad
Calcula subtotal, IVA y total
Muestra el resultado con dos decimales

Siguiente módulo: condicionales y validación con if, else if, else y switch.